Kalman Filter Design and Tuning Document

Author: Vladislav Stefanov

Document versions

Version	Date	Description
1.0	2026-06-08	Initial version
2.0	2026-06-15	Document ready for final submission

Contents

Author: Vladislav Stefanov **1**
What is a Kalman Filter? . . . 2
Overview diagram . . . 3
Notation . . . 3
Part 1 - The 1D case . . . 4
What is being tracked . . . 4
Parameters . . . 4
Step 0 - Initialization . . . 5
Step 1 - Predict state . . . 5
Step 2 - Predict uncertainty . . . 5
Step 3 - Kalman gain . . . 5
Step 4 - Update state . . . 6
Step 5 - Update uncertainty . . . 6
1D cycle summary . . . 6
Example . . . 7
Part 2 - Extending to 6 dimensions . . . 7
State vector . . . 7
Covariance matrix P . . . 7
Step 0 - Initialization . . . 8
Step 1 - Predict state . . . 8
Step 2 - Predict uncertainty . . . 9
Step 3 - Kalman gain . . . 10
Step 4 - Update state . . . 12
Step 5 - Update covariance . . . 12
6D cycle summary . . . 13
Configuration comparison . . . 13
Coasting - no detection available . . . 13
Forward prediction . . . 14
Known limitations . . . 14
Configuration vs runtime variables . . . 14
Parameters set at initialization . . . 14
Variables that change every frame at runtime . . . 15
Key points . . . 15
Appendix A - Matrix transpose . . . 16

List of Figures

1 Overview of the Kalman filter loop as used in this project. Each frame starts from the corrected state of the previous frame. The predict steps (1-2) run unconditionally. If the ZED camera provides a valid detection, the update steps (3-5) correct the state and reduce uncertainty. If no detection is available the filter coasts on the predict step only until a detection resumes. The output is the corrected state for the current frame and the 0.2 s forward extrapolation used for obstacle avoidance. . . . 3

What is a Kalman Filter?

When tracking a moving object, there are two sources of information:

1. **The motion model** - knowing where the object was and how fast it was moving makes it possible to predict where it is now.
2. **The sensor** - the camera measured the object at a certain position, but the measurement is noisy.

Both are imperfect. The Kalman filter is an algorithm that optimally combines these two noisy sources every single frame to produce the best possible estimate of where the object is.

The filter does not just produce an estimate - it also tracks how uncertain it is about that estimate. Every frame, uncertainty grows slightly because the motion model is imperfect, then shrinks when a new camera measurement arrives. The balance between these two forces is controlled automatically.

Note that “camera” and “sensor” are used interchangeably throughout the document.

Overview diagram

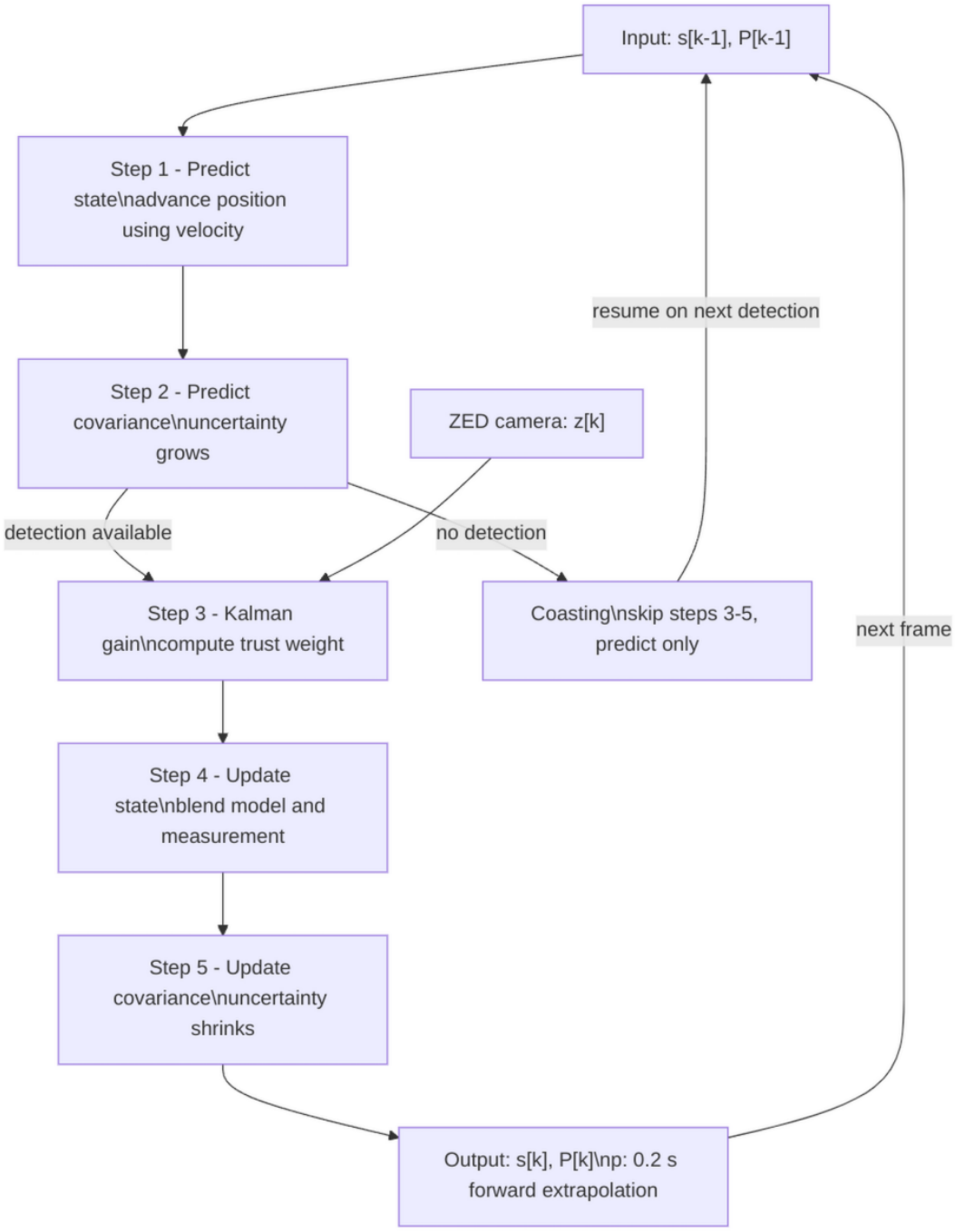


Figure 1: Overview of the Kalman filter loop as used in this project. Each frame starts from the corrected state of the previous frame. The predict steps (1-2) run unconditionally. If the ZED camera provides a valid detection, the update steps (3-5) correct the state and reduce uncertainty. If no detection is available the filter coasts on the predict step only until a detection resumes. The output is the corrected state for the current frame and the 0.2 s forward extrapolation used for obstacle avoidance.

Notation

Standard single-letter notation is used throughout, matching all external references and implementations.

Symbol	Meaning
$\mathbf{s}$	State vector - variables being estimated by the filter (position and velocity)
$\mathbf{P}$	Covariance matrix - variance and covariance of the state estimate error
$\mathbf{F}$	Transition matrix - linear operator that propagates the state forward one timestep under the motion model
$\mathbf{H}$	Observation matrix - linear operator that maps the full state vector to the measurement space
$\mathbf{Q}$	Process noise covariance matrix - variance of the error introduced by the motion model per timestep
$\mathbf{R}$	Measurement noise covariance matrix - variance of the sensor measurement error
$\mathbf{K}$	Kalman gain matrix - optimal weighting factor between the predicted state and the measurement
$\mathbf{z}$	Measurement vector - observed sensor output at frame k
$\mathbf{y}$	Residual vector (innovation) - difference between the measurement and the predicted measurement
$\mathbf{S}$	Innovation covariance matrix - total variance of the residual in measurement space
$\hat{\mathbf{s}}^-$	Predicted state vector - state estimate prior to the measurement update
$\mathbf{P}^-$	Predicted covariance matrix - state error covariance prior to the measurement update
k	Current frame index
$k-1$	Previous frame index
Δt	Time elapsed between consecutive frames (1/30 s at 30 fps)
$^\top$	Matrix transpose operator (see Appendix A)
$^{-1}$	Matrix inverse operator

Part 1 - The 1D case

Before introducing the full 6-variable filter, it helps to understand the same logic applied to a single dimension: the lateral x position of the ball. Velocity is estimated by the full 6D filter (Part 2) - this section uses it to show what happens to the x component. All the same steps apply, but with plain numbers instead of matrices.

What is being tracked

Two quantities are tracked:

- $\hat{x}$ - the estimated x position of the ball (meters)
- P - the variance on that estimate, i.e. how uncertain the position estimate is (m^2)

The standard deviation $\sigma = \sqrt{P}$ gives the uncertainty in the same unit as the position (meters).

Parameters

Before the loop starts, four parameters must be set:

Parameter	Symbol	Unit	What it is	How it is determined
Process noise	Q	m^2	Variance of the motion model error - how wrong the motion model can be per frame. A larger Q means the model is trusted less and the camera measurements get more weight via K.	Design choice. Cannot be measured directly. Reason from physics: how much can velocity change unexpectedly in 33 ms? Tune empirically by testing values and comparing prediction error.
Measurement noise	R	m^2	Variance of the sensor error - how spread out the camera's readings are around the true position. Smaller R means the camera is more precise and gets more weight via K.	Measured from the sensor. Place the object at known positions, record what the sensor reports, compute $R = \sigma^2$. In this project: 45 static captures gave $\sigma_x = 0.008$ m, so $R = 0.000064$ m^2.
Initial uncertainty	P_0	m^2	Variance of the position estimate at the very first detection. Encodes how confident the filter is in the initial state.	Set conservatively large (e.g. $P_0 = 1$ m^2). Exact value does not matter - the filter converges within a few frames. Must not be zero (zero means "perfectly certain," causing all future measurements to be ignored).

Parameter	Symbol	Unit	What it is	How it is determined
Camera measurement	z_k	m	The raw position reading from the sensor at frame k. Input to the update step each frame.	Not set - arrives from the sensor. In this project: ZED depth map sampled inside the YOLO bounding box using front-face clustering.

The mean error is not captured by R. The static experiment also found a mean x error of -1.1 cm - the ZED consistently reports x slightly to the left of the true position. R only models the scatter (random noise), not the offset (systematic bias). The filter will faithfully track the biased measurement. Correcting this requires subtracting the known bias from z_k before feeding it to the filter.

Step 0 - Initialization

On the very first detection, the camera returns a position z_0. State and uncertainty are initialized as:

$$\hat{x}_0 = z_0, \quad P_0 = 1 \text{ m}^2$$

$P_0 = 1$ m² is large on purpose - it says "barely confident yet." The exact value barely matters: it is forgotten within a few frames as long as it is not zero. Zero would mean "perfectly certain," which would make the filter ignore all future measurements.

Step 1 - Predict state

$$\hat{x}_k^- = \hat{x}_{k-1} + v_{k-1} \cdot \Delta t$$

Symbol	Meaning
$\hat{x}_k^-$	predicted x position at frame k, prior to the measurement update
$\hat{x}_{k-1}$	corrected x position from the previous frame
v_{k-1}	x velocity estimate from the previous frame
Δt	time since the last frame = 1/30 s

The hat $\hat{}$ marks an estimated value. The superscript $^-$ marks "prior to incorporating the measurement" - it drops once the update step runs.

Step 2 - Predict uncertainty

$$P_k^- = P_{k-1} + Q$$

Symbol	Meaning
P_k^-	predicted uncertainty at frame k, prior to the measurement update
P_{k-1}	uncertainty from the previous frame
Q	process noise - extra uncertainty added because the model can be wrong

Uncertainty always grows here. It shrinks again once the camera measurement arrives in step 5.

Step 3 - Kalman gain

$$K = \frac{P_k^-}{P_k^- + R}$$

Symbol	Meaning
K	Kalman gain - weight between 0 and 1

Symbol	Meaning
P_k^-	predicted uncertainty (how wrong the model might be)
R	measurement noise (how wrong the camera might be)

K answers: how much should the camera measurement move the estimate?

- $P_k^- \gg R$ (predicted uncertainty much greater than measurement noise): model uncertain, camera reliable $\Rightarrow K \to 1$, camera trusted strongly. - $R \gg P_k^-$ (measurement noise much greater than predicted uncertainty): camera noisy, model confident $\Rightarrow K \to 0$, camera mostly ignored.

Step 4 - Update state

$$y_k = z_k - \hat{x}_k^-, \qquad \hat{x}_k = \hat{x}_k^- + K \cdot y_k$$

Symbol	Meaning
y_k	residual (innovation) - how wrong the model prediction was
z_k	camera measurement of x this frame
$\hat{x}_k^-$	predicted x from step 1
$\hat{x}_k$	corrected x position - final estimate for this frame

The corrected estimate shifts from the prediction toward the measurement by a fraction K of the residual. $K = 1$ jumps fully to the measurement, $K = 0$ ignores it.

Step 5 - Update uncertainty

$$P_k = (1 - K) \cdot P_k^-$$

Symbol	Meaning
P_k	updated uncertainty after incorporating the measurement
K	Kalman gain from step 3
P_k^-	predicted uncertainty from step 2

Seeing a measurement always reduces uncertainty. The factor $(1 - K)$ controls how much: when K is high the filter trusted the camera strongly, so the remaining uncertainty is small. When K is low the camera contributed little new information and uncertainty stays close to P_k^-.

1D cycle summary

The output of each frame ($\hat{x}_k$, P_k) becomes the input to the next frame's step 1. The loop repeats every 33 ms.

Step	Formula	What happens
0. Init	$\hat{x}_0 = z_0,\ P_0 = 1$	set state from first detection
1. Predict state	$\hat{x}_k^- = \hat{x}_{k-1} + v_{k-1} \cdot \Delta t$	advance position by velocity
2. Predict uncertainty	$P_k^- = P_{k-1} + Q$	uncertainty grows
3. Kalman gain	$K = P_k^- \,/\, (P_k^- + R)$	compute trust weight
4. Update state	$\hat{x}_k = \hat{x}_k^- + K \cdot (z_k - \hat{x}_k^-)$	blend model and measurement
5. Update uncertainty	$P_k = (1 - K) \cdot P_k^-$	uncertainty shrinks

Three things to keep in mind:

1. The filter never knows the true position. It always produces an estimate with an attached uncertainty.
2. K is the only thing controlling the balance between model and camera. It is recomputed every frame automatically from P and R.
3. P shrinks when a measurement is incorporated (step 5) and grows when the model runs without one (step 2). It is always bouncing between these two forces.

Example

One full cycle, starting from $\hat{x}_{k-1} = 1.000$ m, $v_{k-1} = 0.600$ m/s, $P_{k-1} = 0.0001$ m², with $Q = 0.0001$ m², $R = 0.000064$ m², and a camera reading $z_k = 1.04$ m.

Step	Calculation	Result
1. Predict state	$1.000 + 0.600 \times \frac{1}{30}$	$\hat{x}_k^- = 1.020$ m
2. Predict uncertainty	$0.0001 + 0.0001$	$P_k^- = 0.0002$ m² ($\sigma \approx 1.4$ cm)
3. Kalman gain	$0.0002/(0.0002 + 0.000064)$	$K \approx 0.758$
4. Update state	$1.02 + 0.758 \times (1.04 - 1.02)$	$\hat{x}_k = 1.035$ m
5. Update uncertainty	$(1 - 0.758) \times 0.0002$	$P_k = 0.0000484$ m² ($\sigma \approx 0.7$ cm)

Reading the result: the model predicted 1.02 m, the camera measured 1.04 m, and the final estimate (1.035 m) lands closer to the camera because K was high. Uncertainty grew from 1.0 cm to 1.4 cm during predict, then shrank to 0.7 cm after the measurement.

Part 2 - Extending to 6 dimensions

In the project, the ball is tracked in 3D space and velocity is also estimated. The state vector has six variables: position (x, y, z) and velocity (v_x, v_y, v_z). The 1D logic is identical - the only change is that scalars become vectors and matrices.

State vector

$$\mathbf{s} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}$$

- (x, y, z) - position of the ball in the camera frame (meters)
- (v_x, v_y, v_z) - velocity of the ball along each axis (meters per second)

Position is observed each frame via the ZED depth map. Velocity is never directly measured - there is no velocity sensor. Instead, the filter infers velocity from the pattern of position corrections over time. Each frame, the residual $\mathbf{y}_k$ (the difference between where the camera sees the ball and where the model predicted it) is distributed across all six state variables by the Kalman gain $\mathbf{K}$. The velocity rows of $\mathbf{K}$ convert position errors into velocity corrections: if the ball consistently appears ahead of the prediction, the filter increases its velocity estimate to close the gap.

The camera coordinate frame used in this project:

- x: lateral (positive = right)
- y: vertical (positive = down)
- z: depth from the camera lens

Covariance matrix P

In 1D, uncertainty was a single number P. With six state variables, uncertainty is a 6×6 matrix. The diagonal entries are the variances on each state variable, equivalent to the single P from 1D:

$$\mathbf{P} = \begin{bmatrix} P_{xx} & P_{xy} & P_{xz} & P_{xv_x} & P_{xv_y} & P_{xv_z} \\ \cdots & P_{yy} & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & P_{zz} & \cdots & \cdots & \cdots \\ P_{v_x x} & \cdots & \cdots & P_{v_x v_x} & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & P_{v_y v_y} & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & P_{v_z v_z} \end{bmatrix}$$

The matrix is called a covariance matrix because the off-diagonal entries are covariances - they measure how two different variables vary together. For example, P_{xv_x} captures whether an error in the x position estimate tends to come with an

error in the v_x estimate. The matrix is symmetric, so $P_{xv_x} = P_{v_x x}$ - "x error tends to come with v_x error" is the same relationship as "v_x error tends to come with x error." These entries are computed automatically by the filter and are not set manually.

Large diagonal value = uncertain, small diagonal value = confident. A large diagonal entry means the filter is uncertain about its own prediction, so it will trust the camera more for that variable (higher K). A small diagonal entry means the filter is confident in its prediction, so it will weight the camera less.

Step 0 - Initialization

On the first detection, the camera provides a 3D position. The state is initialized with that position and zero velocity:

$$\mathbf{s}_0 = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Zero initial velocity is an assumption - there is no information to estimate it from a single frame. The filter converges toward the real velocity after the first few measurement-to-measurement intervals. This is why predictions are suppressed until at least two corrections have been applied.

Initial uncertainty is set to the 6×6 identity matrix:

$$\mathbf{P}_0 = \mathbf{I}_{6\times 6} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This expresses moderate, uniform uncertainty about all six variables. The off-diagonal entries are zero because at initialization there is only one detection and no history of errors - the filter has no evidence that errors in different variables are correlated. After the first predict step, $\mathbf{F}$ couples position and velocity and the off-diagonals become nonzero. The exact values on the diagonal matter little - they are forgotten within a few frames as measurements arrive.

Step 1 - Predict state

Transition matrix F

$\mathbf{F}$ encodes the constant-velocity motion model: *new position = old position + velocity × time.* Velocity is assumed unchanged.

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \Delta t = \tfrac{1}{30}\ \text{s}$$

The top-right 3×3 block (containing Δt) propagates velocity into position. The bottom-right 3×3 identity block carries velocity forward unchanged.

Prediction calculation

The predicted state is computed by multiplying the transition matrix $\mathbf{F}$ by the previous state:

$$\hat{\mathbf{s}}_k^- = \mathbf{F} \cdot \hat{\mathbf{s}}_{k-1}$$

Symbol	Meaning
$\hat{\mathbf{s}}_k^-$	predicted state at frame k, prior to the measurement update (6×1 vector)
$\mathbf{F}$	transition matrix - encodes the constant-velocity motion model (6×6 matrix)
$\hat{\mathbf{s}}_{k-1}$	corrected state from the previous frame (6×1 vector)

Expanding $\mathbf{F} \cdot \hat{\mathbf{s}}_{k-1}$ shows what the model predicts:

$$\hat{\mathbf{s}}_k^- = \mathbf{F} \cdot \hat{\mathbf{s}}_{k-1} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} x + v_x \Delta t \\ y + v_y \Delta t \\ z + v_z \Delta t \\ v_x \\ v_y \\ v_z \end{bmatrix}$$

Each position advances by its corresponding velocity times the time step. Velocity is copied forward unchanged - this is the constant-velocity assumption.

Step 2 - Predict uncertainty

The covariance update in 1D was $P_k^- = P_{k-1} + Q$. In 6D, the uncertainty must also be propagated through the motion model using $\mathbf{F}$:

$$\mathbf{P}_k^- = \mathbf{F} \cdot \mathbf{P}_{k-1} \cdot \mathbf{F}^\top + \mathbf{Q}$$

Symbol	Meaning
$\mathbf{P}_k^-$	predicted covariance at frame k, prior to the measurement update (6×6 matrix)
$\mathbf{F}$	transition matrix (6×6)
$\mathbf{P}_{k-1}$	covariance from the previous frame (6×6 matrix)
$\mathbf{F}^\top$	transpose of $\mathbf{F}$ (6×6)
$\mathbf{Q}$	process noise matrix (6×6)

The $FPF^\top$ form is the standard rule for transforming a covariance through a linear model - a state transforms as Fs, but its covariance transforms as $FPF^\top$.

Why $\mathbf{FPF}^\top$ and not just $\mathbf{P} + \mathbf{Q}$? The state variables are coupled - position advances using velocity. When $\mathbf{F}$ transforms the state, it also mixes the uncertainties of position and velocity together. The formula $\mathbf{FPF}^\top$ applies $\mathbf{F}$ from both sides to correctly propagate all six uncertainties and their cross-correlations forward. For the x axis specifically, this gives:

$$P_{xx}^- = P_{xx} + 2\Delta t \cdot P_{xv_x} + \Delta t^2 \cdot P_{v_x v_x}$$

The position uncertainty grows not just by Q but also by $\Delta t^2 \cdot P_{v_x v_x}$ - **the more uncertain the velocity estimate is, the faster position uncertainty inflates during predict.**

Why +Q? $\mathbf{FPF}^\top$ only accounts for uncertainty already present from last frame. It does not account for the motion model being wrong. $\mathbf{Q}$ adds that missing uncertainty back in, preventing the filter from becoming overconfident in its own predictions.

Process noise matrix Q

$$\mathbf{Q} = \begin{bmatrix} 0.0001 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.0001 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.0001 & 0 & 0 & 0 \\ 0 & 0 & 0 & q_v & 0 & 0 \\ 0 & 0 & 0 & 0 & q_v & 0 \\ 0 & 0 & 0 & 0 & 0 & q_v \end{bmatrix}$$

Q is diagonal - no cross-axis correlations are assumed in the model error.

Position rows use 0.0001 m² ($\sigma = 1$ cm): position changes predictably via **F** between frames 33 ms apart.

Velocity rows use q_v: velocity can change unexpectedly if the ball accelerates, decelerates or changes direction. Two configurations were evaluated:

Configuration	q_v	$\sigma_v = \sqrt{q_v}$	Effect
`baseline`, `zed_noise`	0.01	0.10 m/s	slow velocity adaptation
`high_vel`, `zed_noise_high_vel`	0.10	0.316 m/s	faster velocity adaptation

Both non-baseline values of q_v produced equivalent prediction accuracy (mean ADE 1.3 cm, mean FDE 2.1 cm), each better than `baseline`. This suggests the constant-velocity model assumption itself was the main ceiling, not the choice of q_v.

Intuition

Predicting forward always makes the filter less certain. $\mathbf{FPF}^\top$ carries the old uncertainty forward and inflates position with any velocity error, while Q adds the uncertainty from the motion model being imperfect. Step 2 is the "uncertainty grows" half of the cycle, deliberately widening the estimate so the next measurement (Step 3) gets appropriate weight. The result is the calculated uncertainty $\mathbf{P}_k^-$ which is a 6×3 matrix.

Step 3 - Kalman gain

The Kalman gain in 1D was a scalar K. In 6D it becomes a 6×3 matrix **K**. It answers the same question: for each meter of error in the 3D measurement, how much should each of the six state variables be corrected?

The full formula is:

$$\mathbf{K} = \mathbf{P}_k^- \cdot \mathbf{H}^\top \cdot \left(\mathbf{H} \cdot \mathbf{P}_k^- \cdot \mathbf{H}^\top + \mathbf{R}\right)^{-1}$$

This is built in steps. First, the observation matrix **H** must be introduced.

Observation matrix H

The camera measures position only - not velocity. The state vector has six variables but the measurement has only three. **H** is a 3×6 matrix that extracts the position rows from the full state:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Multiplying **H** by the full state vector picks out only the position entries:

$$\mathbf{H} \cdot \hat{\mathbf{s}}_k^- = \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{z} \end{bmatrix}$$

This is the predicted position expressed in the same 3D space as the camera measurement, so the two can be compared directly.

Building the Kalman gain step by step

Piece 1: $\mathbf{H} \cdot \mathbf{P}_k^-$

$\mathbf{H}$ is 3×6 and $\mathbf{P}_k^-$ is 6×6. Result is 3×6.

This extracts only the rows of $\mathbf{P}_k^-$ that correspond to position - which parts of the total uncertainty are visible to the sensor.

Piece 2: $\mathbf{H} \cdot \mathbf{P}_k^- \cdot \mathbf{H}^\top$

Take piece 1 (3×6) and right-multiply by $\mathbf{H}^\top$ (6×3). Result is 3×3.

This also extracts the relevant columns, leaving only the position-to-position block of the covariance:

$$\mathbf{H} \cdot \mathbf{P}_k^- \cdot \mathbf{H}^\top = \begin{bmatrix} P_{xx} & P_{xy} & P_{xz} \\ P_{yx} & P_{yy} & P_{yz} \\ P_{zx} & P_{zy} & P_{zz} \end{bmatrix}$$

This is the prediction uncertainty expressed in the same 3D space as the measurement.

Intuition

Because the camera only measures position, the prediction uncertainty must be reduced from its full 6D form to the 3D position space before it can be compared with the measurement. $HP_k^-H^\top$ performs this projection, discarding the velocity-related uncertainty and leaving a 3x3 matrix that matches R in both dimension and units, so the two combine cleanly into S.

Piece 3: $\mathbf{S} = \mathbf{H} \cdot \mathbf{P}_k^- \cdot \mathbf{H}^\top + \mathbf{R}$

Add the measurement noise matrix $\mathbf{R}$ (also 3×3) to piece 2 (see below what is $\mathbf{R}$). Result is 3×3.

$$\mathbf{S} = \begin{bmatrix} P_{xx} + R_x & P_{xy} & P_{xz} \\ P_{yx} & P_{yy} + R_y & P_{yz} \\ P_{zx} & P_{zy} & P_{zz} + R_z \end{bmatrix}$$

$\mathbf{S}$ is the innovation covariance - the total uncertainty in measurement space combining prediction uncertainty and sensor noise. This is the 6D equivalent of the denominator $P_k^- + R$ in the 1D gain formula.

Piece 4: $\mathbf{S}^{-1}$

Invert $\mathbf{S}$. If $\mathbf{S}$ is large (high total uncertainty), $\mathbf{S}^{-1}$ is small, producing a smaller gain - the correction is less aggressive when total uncertainty is high. This is the matrix equivalent of dividing by $P_k^- + R$ in the 1D case.

Piece 5: $\mathbf{P}_k^- \cdot \mathbf{H}^\top$

$\mathbf{P}_k^-$ is 6×6 and $\mathbf{H}^\top$ is 6×3. Result is 6×3.

This gives the first three columns of $\mathbf{P}_k^-$ - how each of the six state variables co-varies with the three measured position variables. Crucially, this includes the velocity rows: $P_{v_x x}$, $P_{v_x y}$, $P_{v_x z}$ and so on. These cross-terms are what allow the filter to correct velocity estimates from position measurements, even though velocity is never directly observed.

Final: multiply pieces 5 and 4

$$\mathbf{K} = \underbrace{\mathbf{P}_k^- \cdot \mathbf{H}^\top}_{6\times3} \cdot \underbrace{\mathbf{S}^{-1}}_{3\times3} = \underbrace{\mathbf{K}}_{6\times3}$$

Measurement noise matrix R

$\mathbf{R}$ is 3×3 and diagonal - measurement errors in x, y and z are assumed independent:

$$\mathbf{R} = \begin{bmatrix} R_x & 0 & 0 \\ 0 & R_y & 0 \\ 0 & 0 & R_z \end{bmatrix}$$

Values for the two configurations evaluated in this project:

Configuration	R_x	R_y	R_z	Source
`baseline`	0.001 m²	0.001 m²	0.001 m²	arbitrary default ($\sigma \approx 3.2$ cm)
`zed_noise`	0.000064 m²	0.000121 m²	0.000064 m²	measured from static ZED experiment

The `zed_noise` values are derived from the static accuracy experiment: $\sigma_x = 0.8$ cm, $\sigma_y = 1.1$ cm, $\sigma_z = 0.8$ cm, converted to variance. The `baseline` values over-trust the motion model because they tell the filter the camera is approximately four times noisier than it actually is.

Step 4 - Update state

The measurement vector $\mathbf{z}_k$ is the 3D position returned by the ZED camera this frame:

$$\mathbf{z}_k = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

The residual is computed first:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H} \cdot \hat{\mathbf{s}}_k^-$$

Symbol	Meaning
$\mathbf{y}_k$	residual (3×1 vector) - how wrong the model prediction was in each position axis
$\mathbf{z}_k$	camera measurement (3×1 vector)
$\mathbf{H} \cdot \hat{\mathbf{s}}_k^-$	predicted position extracted from the full predicted state (3×1 vector)

If the model was perfect, $\mathbf{y}_k = \mathbf{0}$.

The corrected state is then:

$$\hat{\mathbf{s}}_k = \hat{\mathbf{s}}_k^- + \mathbf{K} \cdot \mathbf{y}_k$$

$\mathbf{K}$ is 6×3 and $\mathbf{y}_k$ is 3×1, so $\mathbf{K} \cdot \mathbf{y}_k$ is 6×1 - a correction vector applied to all six state variables. Velocity also gets corrected here even though it was never measured, because the velocity rows of $\mathbf{K}$ (derived from the cross-terms in piece 5 above) distribute the position residual into a velocity correction.

Step 5 - Update covariance

After incorporating the measurement, uncertainty decreases:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K} \cdot \mathbf{H}) \cdot \mathbf{P}_k^-$$

Symbol	Meaning
$\mathbf{P}_k$	updated covariance after the camera measurement (6×6 matrix)
$\mathbf{I}$	6×6 identity matrix

Symbol	Meaning
$\mathbf{K}$	Kalman gain (6×3 matrix)
$\mathbf{H}$	observation matrix (3×6 matrix)
$\mathbf{P}_k^-$	predicted covariance from step 2 (6×6 matrix)

$\mathbf{K} \cdot \mathbf{H}$ is 6×6. $\mathbf{I} - \mathbf{K} \cdot \mathbf{H}$ is a scaling matrix with values between 0 and 1 that shrinks $\mathbf{P}_k^-$. A larger $\mathbf{K}$ (more trust in the camera) produces a larger reduction in uncertainty. This is the direct 6D equivalent of $P_k = (1 - K) \cdot P_k^-$ from 1D.

6D cycle summary

Step	Formula	What happens
Init	$\mathbf{s}_0 = [x, y, z, 0, 0, 0]^\top,\ \mathbf{P}_0 = \mathbf{I}$	set state from first detection
Predict state	$\hat{\mathbf{s}}_k^- = \mathbf{F} \cdot \hat{\mathbf{s}}_{k-1}$	advance state by motion model
Predict covariance	$\mathbf{P}_k^- = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^\top + \mathbf{Q}$	uncertainty grows
Residual	$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{s}}_k^-$	how wrong was the prediction?
Innovation covariance	$\mathbf{S} = \mathbf{H}\mathbf{P}_k^-\mathbf{H}^\top + \mathbf{R}$	total uncertainty in measurement space
Kalman gain	$\mathbf{K} = \mathbf{P}_k^-\mathbf{H}^\top\mathbf{S}^{-1}$	compute trust weight
Update state	$\hat{\mathbf{s}}_k = \hat{\mathbf{s}}_k^- + \mathbf{K}\mathbf{y}_k$	blend model and measurement
Update covariance	$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_k^-$	uncertainty shrinks

---

Configuration comparison

Four configurations were evaluated, varying two parameters independently: q_v (velocity process noise) and $\mathbf{R}$ (measurement noise).

Configuration	q_v	R_x (m²)	R_y (m²)	R_z (m²)
`baseline`	0.01	0.001	0.001	0.001
`zed_noise`	0.01	0.000064	0.000121	0.000064
`high_vel`	0.10	0.001	0.001	0.001
`zed_noise_high_vel`	0.10	0.000064	0.000121	0.000064

The `baseline` R values are an arbitrary default ($\sigma \approx 3.2$ cm per axis). The `zed_noise` R values come from the static ZED accuracy experiment: 45 known-position captures gave $\sigma_x = 0.8$ cm, $\sigma_y = 1.1$ cm, $\sigma_z = 0.8$ cm, converted to variance ($R = \sigma^2$).

$q_v = 0.01$ ($\sigma_v = 0.10$ m/s) represents slow velocity adaptation. $q_v = 0.10$ ($\sigma_v = 0.316$ m/s) allows the filter to respond faster to velocity changes.

The two higher-q_v configurations (`high_vel`, `zed_noise_high_vel`) both achieved mean ADE 1.3 cm and mean FDE 2.1 cm, each better than `baseline`. The near-identical results between them suggest the accuracy ceiling was the constant-velocity model assumption, not the choice of q_v or $\mathbf{R}$.

Coasting - no detection available

If YOLO misses the ball, no measurement is available. The update step (steps 3-5) is skipped entirely. Only the predict step runs - the filter keeps advancing using its current velocity estimate. This is called coasting.

During coasting, $\mathbf{P}$ keeps growing each frame with no correction, so the filter becomes progressively less confident. Predictions are still issued during coasting.

If detection fails for more than `MAX_MISSED = 5` consecutive frames, prediction output is suppressed and the filter resets on the next detection. The YOLO confidence threshold is set to 0.15 (below the default of 0.25) to reduce missed detections and minimize the frequency of coasting.

Forward prediction

After the update step, the filter extrapolates the current state forward 0.2 seconds to produce a predicted future position. This is done by simple linear extrapolation - no additional filter steps:

$$\hat{\mathbf{p}}(h) = \begin{bmatrix} x + v_x \cdot h \\ y + v_y \cdot h \\ z + v_z \cdot h \end{bmatrix}, \quad h \in \left\{ \tfrac{1}{30}, \tfrac{2}{30}, \tfrac{3}{30}, \tfrac{4}{30}, \tfrac{5}{30}, \tfrac{6}{30} \right\} \text{ s}$$

Symbol	Meaning
$\hat{\mathbf{p}}(h)$	predicted 3D position at horizon h seconds ahead
x, y, z	current position from the filter state after the update step
v_x, v_y, v_z	current velocity from the filter state after the update step
h	prediction horizon in seconds

The final step at $h = 0.2$ s is the primary output (the red circle in the annotated video). Six steps are computed per frame at 33 ms intervals leading up to 0.2 s.

Why linear extrapolation and not repeated predict steps? Running $\mathbf{F}$ forward six times would advance the covariance matrix $\mathbf{P}$, corrupting the filter's uncertainty estimate for the next real measurement. Under the constant-velocity model, linear extrapolation gives the identical predicted position while leaving $\mathbf{P}$ unchanged.

Known limitation: the extrapolation does not propagate uncertainty. The predicted position has no confidence interval attached. In reality, prediction uncertainty grows with the horizon length, but this is not reflected in the output.

Predictions are only issued after `corrections >= 2` - ensuring at least one measurement-to-measurement interval has elapsed so the velocity estimate is meaningful before it is extrapolated.

Known limitations

- **Constant-velocity model** - the filter does not model acceleration, gravity or bouncing. Curved trajectories or sudden velocity changes cause prediction lag. Straight and diagonal throws are handled well; bounces and rapid direction changes are not.
- **No uncertainty on forward prediction** - the predicted position has no confidence interval. Uncertainty grows with the prediction horizon but is not propagated or reported.
- **Single-object tracking** - one filter instance tracks one object at a time. Multi-object tracking requires data association across frames and one independent filter instance per tracked object.
- **Ground truth from the same pipeline** - the evaluation ground truth is produced by the same YOLO and ZED pipeline that drives predictions. Systematic errors in detection or depth cancel between the prediction and the future ground truth frame. ADE and FDE likely underestimate true prediction error relative to real-world positions.
- **ZED accuracy experiment coverage** - the static accuracy experiment was conducted at one height ($y = 0.28$ m) and depths up to 0.77 m. Biases at other heights or longer distances are not characterized.
- **Bias not corrected** - the ZED shows a mean y error of -5.0 cm and a mean z error of $+4.1$ cm. These are stable and could be corrected with fixed offsets applied to $\mathbf{z}_k$ before the update step, but this was not applied in the pipeline.

Configuration vs runtime variables

This section separates the quantities that are **set once at initialization** (configuration) from those the filter **computes every frame** (runtime). A common misunderstanding is treating the Kalman gain K as a tunable parameter - it is not. The only knobs are Q, R and the initial state/uncertainty, everything else is derived.

Parameters set at initialization

These are fixed before the loop starts and do not change during a run.

Parameter	Symbol	How it is set	Effect of changing it
Process noise	Q	Design choice. Position rows fixed at 0.0001 m^2. Velocity rows set via q_v (0.01 or 0.10 in this project).	Larger Q trusts the motion model less, so measurements get more weight via K. Raising q_v lets velocity adapt faster to direction changes.
Measurement noise	R	Measured from the static ZED experiment ($R = \sigma^2$ per axis), or an arbitrary default in the baseline config.	Smaller R trusts the camera more (higher K). Over-large R over-trusts the model and lags real motion.
Initial state	s_0	First detection sets position; velocity initialized to zero.	Zero-velocity start causes a few frames of convergence before predictions are meaningful (hence predictions suppressed until corrections >= 2).
Initial uncertainty	P_0	Set to the 6x6 identity matrix I.	Exact values matter little - forgotten within a few frames. Must not be zero (zero = "perfectly certain", which makes the filter ignore all measurements).
Transition matrix	F	Built from $\Delta t = 1/30$ s and the constant-velocity model.	Fixed structure. Only changes if Δt or the motion model changes.
Observation matrix	H	Fixed - extracts the three position rows from the state.	Constant; the camera always measures the same three position variables.
Timestep	Δt	Fixed at 1/30 s (30 fps).	Assumed constant. If real frame intervals jitter, Δt and therefore F would need to be recomputed per frame.

In this project, F, H, Q, R and Δt are treated as constant for the entire run. In a general Kalman filter they may be made time-varying (for example Δt tracking actual inter-frame timing, or R adapting with range), but that is out of scope here.

Variables that change every frame at runtime

These are recomputed each cycle. They are outputs of the filter, not configuration.

Variable	Symbol	When it changes
State estimate	s / $\hat{s}_k^-$	Predict step shifts it forward; update step corrects it toward the measurement.
Covariance	P / P_k^-	Grows in the predict step (Step 2), shrinks in the update step (Step 5).
Measurement	z_k	New camera reading each frame (external input).
Residual	y_k	Difference between measurement and prediction - new each frame.
Innovation covariance	S	Derived from P_k^- and R; recomputed each frame.
Kalman gain	K	Derived from P_k^- and S; recomputed each frame from the current uncertainty.

Key points

1. K **is derived, not tuned.** Q and R are the knobs. K is computed every frame from the live uncertainty P and the configured R. As P changes, K changes automatically.
2. P **evolves even without a measurement.** During coasting (no detection), P keeps growing via the predict step with no update to shrink it, so the filter becomes progressively less confident. K, y and S are only recomputed when an update actually runs.
3. **Mental model:**
   - You configure: F, H, Q, R, s_0, P_0 (constant during the run).
   - The filter computes: s, P, and from those K, y, S (changing).

- The camera supplies: z (changing, external input).

Appendix A - Matrix transpose

Transposing a matrix flips it so rows become columns:

$$\mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v}^\top = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

For a 2×3 matrix, transposing produces a 3×2 matrix:

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \quad \Rightarrow \quad \mathbf{A}^\top = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}$$

In the Kalman filter, $^\top$ appears in $\mathbf{FPF}^\top$ and $\mathbf{HP}^-\mathbf{H}^\top$. In both cases it ensures the result remains a valid symmetric covariance matrix - a requirement for $\mathbf{P}$ to be mathematically well-formed.